Problem Summary

- Given the roots of a binary tree, determine whether it is a valid Binary Search Tree.

A valid BST means:

→ every node in the left subtree is smaller than the current node

→ every node in the right subtree is greater than the current node

→ both subtrees must also be valid BSTs

Key Idea

It is not enough to compare a node with its direct children.

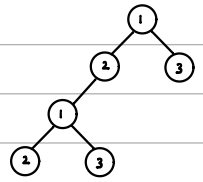
Each node must stay within a valid range:

→ left subtree nodes must be less than an upper bound

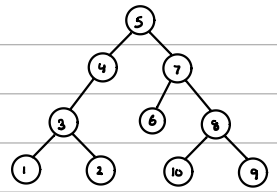
→ right subtree nodes must be greater than an lower bound

Diagram Examples

Not Valid Tree:



Valid:



Approach

→ DFS Recursion

1) Start from root with range $(-\infty, \infty)$

2) For each node:

→ if node is missing, return True

→ if node is outside allowed range, return False

3) Recursively validate:

→ left subtree with updated upper bound

→ right subtree with updated lower bound

4) Return True only if both sides are valid

Time Complexity: $O(n)$ / $O(\log n)$ → Each node is visited once
(worst case / best case)

Space Complexity: $O(h)$ → Recursion stack, where h is tree height